

Predicting Scalability Limits and MVCC Failures in Hyperledger Fabric Supply Chains using Machine Learning

Dhanish Jose and Pradeepmon T G

Department of Mechanical Engineering, National Institute of Technology
Calicut, India

Abstract

Permissioned blockchain networks such as Hyperledger Fabric are increasingly being used in agricultural supply chains to improve transparency and facilitate smart contract-driven automated payments and complex transactions. However, one of the main limitations to the seamless execution of transactions is the multi-version concurrency conflicts arising from the consensus mechanism of Fabric. MVCC conflicts stem from the concurrent high-frequency operations involving shared keys in the smart contract and cannot be eliminated. One possible method is to regulate the transaction rate by predicting the failures and throughput that can occur for a particular load and participant density to which the system is subjected. This study focuses on predicting MVCC conflicts during high-frequency transactions across different loads and participant counts in the blockchain system. By predicting the likelihood of MVCC conflicts, the system can adjust transaction load to mitigate them during transactions. The primary research objective is to develop a machine learning-based predictive framework—using a Random Forest ensemble—capable of predicting transaction throughput and MVCC failure rates from static algorithmic complexity features and dynamic network metrics. The trained Random Forest model demonstrated robust generalization, achieving a testing R^2 of 0.93 for throughput prediction and 0.96 for MVCC failure rate prediction. The feature importance scores generated by the trained models mathematically prove that while transaction failures are distributed across multiple network stress factors, raw network load dominates the maximum achievable throughput (87.8% predictive importance). These findings provide a generalizable, interpretable framework for predicting and diagnosing performance bottlenecks in permissioned blockchain deployments.

Keywords

Hyperledger Fabric; MVCC conflict prediction; Blockchain performance analysis; Random Forest; Transaction throughput; Failure rate prediction; Smart contracts; Agricultural supply chain; Hyperledger Caliper; Permissioned blockchain

1 Introduction

Permissioned blockchain frameworks, particularly Hyperledger Fabric, are increasingly used to build trustworthy, multi-stakeholder supply chain systems. Their inherent ability to record transactions immutably, enforce role-based access control, and automate transactions using smart contracts makes them suitable for high-value agricultural supply chain systems. Traditional agricultural supply chains are grappling with a range of challenges, including a lack of transparency, opaque auctioning, and delayed payments. Integrating blockchain into a traditional agricultural supply chain can effectively address many of these challenges, making it a compelling case study for blockchain-based systems.

Hyperledger Fabric, in particular, has been adopted in numerous agri-food traceability initiatives by organizations such as the World Wide Fund (WWF) and the Food and Agriculture Organization of the United Nations (FAO) (World Wildlife Fund (WWF) (2019)), yet comprehensive systems addressing complex auction and settlement concurrency remain scarce in regional agricultural supply chains. Existing Fabric implementations are limited to traceability, and only a few studies have focused on proper benchmarking to assess their scalability and real-world performance. In Hyperledger Fabric’s optimistic concurrency model, if two transactions read the same ledger key version within the same block window, only one can commit; the other is silently dropped as an MVCC `READ_CONFLICT` failure. These conflicts often cause the concurrent transactions to fail, restrict the deployment of the fabric system in the real world, and limit its scalability. Existing studies focusing on benchmarking and performance optimization often overlook this critical bottleneck in the Fabric-based blockchain system/

This paper addresses this gap by predicting MVCC failures in a blockchain-based system under different transaction loads and network metrics. A Hyperledger Fabric-based agricultural supply chain platform that integrates end-to-end lot traceability, a smart contract-driven, decentralized auction mechanism for transparent price discovery, and automated financial settlements is selected for the proposed study. Crucially, this specific supply chain architecture was selected because it serves a dual purpose: it addresses an urgent socio-economic need while providing an ideal technical testbed for performance modeling. Deploying such a platform—with its high-frequency auction bidding, concurrent payment settlements, and multiple participant roles—creates massive “hot-key” contention. This inherently exposes the network to severe Multi-Version Concurrency Control (MVCC) conflicts under load. The complex business logic of agricultural auction use cases naturally generates the exact high-contention scenarios required to rigorously study and model these structural failure modes.

Predicting and diagnosing these MVCC failure rates and throughput ceilings analytically is challenging because the relationship between concurrent participant density, chaincode complexity, and failure probability is highly non-linear. Existing literature on Hyperledger Fabric performance benchmarking primarily characterizes behavior under controlled settings and rarely develops generalizable predictive models. This study addresses this gap directly: the **primary objective** is to develop and validate a machine learning-based predictive framework—using a Random Forest ensemble—that can accurately forecast MVCC failure rates and transaction throughput from interpretable structural features (ledger reads/writes, hot-key participant density, send rate, and Caliper workers). A representative agricultural

supply chain system serves as the empirical testbed, providing a rich dataset of benchmarked chaincode functions under a comprehensive matrix of transaction loads and participant configurations.

2 Related Work

Agri-food supply chains involve the movement of agricultural products from producers to consumers through several stakeholders, including farmers, processors, distributors, retailers, and consumers (Weerabahu et al., 2022; Nayal et al., 2021). Blockchain technology has gained attention as a tool for improving trust, traceability, and accountability in these supply chains (Zhao et al., 2019; Shahid et al., 2020).

2.1 Hyperledger-Based Applications in Agricultural Supply Chains

Initial research utilizing Hyperledger Fabric has predominantly focused on the architectural feasibility of permissioned networks to ensure end-to-end traceability for food safety and regulatory compliance Marchese and Tomarchio (2022); Zhang et al. (2022); Iftekhhar and Cui (2021); Joo and Han (2021). While these foundational works successfully establish the infrastructure for secure data sharing, they frequently operate in siloed or controlled environments that fail to account for the dynamic, multi-stakeholder nature of real-world agri-food ecosystems. A significant research gap persists in transitioning passive traceability models into active, transactional platforms capable of supporting automated financial logic, incentive mechanisms, and fair price discovery. Recent state-of-the-art systems Rani et al. (2025); Talib et al. (2026); Zhang et al. (2023) introduce dynamic pricing and IPFS storage, but often lack specialized MVCC conflict resolution for high-frequency bidding or explicitly identify concurrency issues as unresolved bottlenecks.

2.2 Performance Benchmarking of Blockchain Systems

Prior works have predominantly employed benchmarking tools such as Hyperledger Caliper to analyze throughput and latency Kumar et al. (2026); Kaushal et al. (2025); Thakkar et al. (2018); El Hajji et al. (2024). However, despite these contributions, several limitations remain evident. Most studies focus on moderate transaction rates or controlled environments and do not perform stress testing under high transaction loads or distributed network conditions. Furthermore, existing benchmarking efforts largely overlook application-specific transaction patterns, particularly those involving high-frequency financial interactions such as auctions. A critical gap is the lack of analysis of concurrency control mechanisms in Hyperledger Fabric, especially Multi-Version Concurrency Control (MVCC), which significantly impacts transaction validity under concurrent workloads. Existing works either ignore MVCC conflicts or do not propose system-level optimizations to mitigate them. To address these limitations, the proposed study adopts a Caliper-driven benchmarking framework to evaluate a cardamom auction system under high transaction rates, serving as the empirical foundation for subsequent machine learning analysis.

2.2.1 MVCC Conflict Management Challenges

Multi-Version Concurrency Control (MVCC) ensures transaction isolation in Hyperledger Fabric but introduces significant performance bottlenecks because its Execute-Order-Validate (EOV) architecture defers conflict detection to the final validation phase, wasting resources on aborted transactions Stolidis et al. (2024). To mitigate this, literature proposes early detection mechanisms (e.g., OEMVCC, SyncMap) that shift MVCC checks to the ordering phase Stolidis et al. (2024); Trabelsi and Zhang (2023). Further optimizations include fine-grained distributed locking and cache-based transaction reordering to minimize contention prior to validation, as well as transaction repair mechanisms utilizing dependency graphs to partially re-execute conflicting operations rather than aborting them entirely.

2.2.2 Machine Learning for Performance Prediction

To address performance bottlenecks beyond architectural modifications, researchers are increasingly utilizing Machine Learning (ML) to predict conflicts and optimize execution. Supervised learning methods, such as Random Forest regression, have proven highly effective at capturing non-linear system dependencies to accurately forecast critical metrics like throughput and latency using benchmarking data from tools like Hyperledger Caliper Dewi (2025). Furthermore, AI-driven dynamic protocol adaptation leverages reinforcement learning and Bayesian optimization to intelligently tune block intervals and sizes, actively alleviating scalability bottlenecks Pawar and Patil (2025). Beyond core consensus tuning, reinforcement learning algorithms like Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) optimize transaction distribution and dynamic sharding, intelligently allocating loads based on real-time arrival rates to minimize cross-shard communication overhead Devi et al. (2025). These advancements share conceptual similarities with predictive resource allocation architectures in broader cloud computing environments, facilitating the scalable and adaptive deployment of decentralized applications.

2.2.3 Identified Research Gaps

Despite these advancements, critical gaps remain in the operational deployment of Fabric networks. Existing MVCC management strategies typically require modifying Fabric’s core consensus and validation pipeline, introducing substantial deployment overhead. Conversely, while predictive ML models like HFGIN offer powerful structural conflict detection, their reliance on runtime Graph Neural Networks introduces heavy computational overhead during the critical path of transaction processing. There is a distinct lack of lightweight, application-layer predictive models that can anticipate MVCC bottlenecks pre-deployment based on static chaincode characteristics. This study addresses this gap by proposing a pre-deployment Random Forest regression model that predicts failure rates and latency based on chaincode complexity and participant density, completely avoiding runtime consensus overhead.

3 System Architecture and Implementation

This study follows a design-and-evaluation methodology. First, a Hyperledger Fabric-based agricultural supply chain platform was designed to support traceability, quality testing, auction-based price discovery, smart contract-driven payments, and packet-level consumer verification. Second, the system was implemented using JavaScript chaincode, a React.js frontend, a Node.js REST API layer, and a multi-organization Fabric network. Finally, the system was functionally validated and then comprehensively benchmarked using Hyperledger Caliper under a wide range of transaction loads and participant configurations to generate the empirical dataset for machine learning-based MVCC and throughput prediction.

3.1 System Overview

The proposed system is a permissioned blockchain-based platform for a high-value agricultural supply chain. It connects five stakeholder groups: farmers, aggregators, retailers, consumers, and the bank. Farmers submit crop lots with production details, and aggregators test and grade the lots for quality compliance. Approved lots are offered to retailers through a smart contract-driven bidding process. After purchase, retailers pack the lots into traceable packets, while consumers verify the packet history using the packet code. The bank creates and funds internal wallets to support smart contract-driven payments. All key events are recorded on the ledger to ensure transparency, accountability, and end-to-end traceability. This architecture—with its concurrent financial transactions, competing auction bids, and shared wallet state—creates MVCC contention patterns that serve as the primary subject of the performance analysis in this study.

3.2 Blockchain Network Architecture

The blockchain network was implemented using Hyperledger Fabric with five organizations: FarmersMSP, AggregatorsMSP, RetailersMSP, ConsumersMSP, and BankMSP. Each organization was configured with its own peer, MSP, and certificate authority. The organizations interact through a common channel, while CouchDB stores the world state. The network was created by extending the Fabric test network using customized Docker Compose, crypto-configuration, and channel configuration files. Raft was used as the ordering mechanism.

3.3 Smart Contract Workflow

The business logic was implemented as JavaScript chaincode. The bank creates and funds wallets using `createAccount`. Farmers submit lots via `createLot`, and aggregators update quality status using `testCardamom`. Retailers bid via `makeOffer`, and the highest bidder executes `purchaseLot`, transferring funds automatically from their wallet to the farmer’s wallet. Retailers repackage purchased lots using `createPacket`. Key features include: `createWallet()` and `depositMoney()`. Farmers submit agricultural lots using `submitProduce()`, and the testing fee is transferred to the selected aggregator. Aggregators record quality test results and grading details using `testCardamom()`. Approved lots

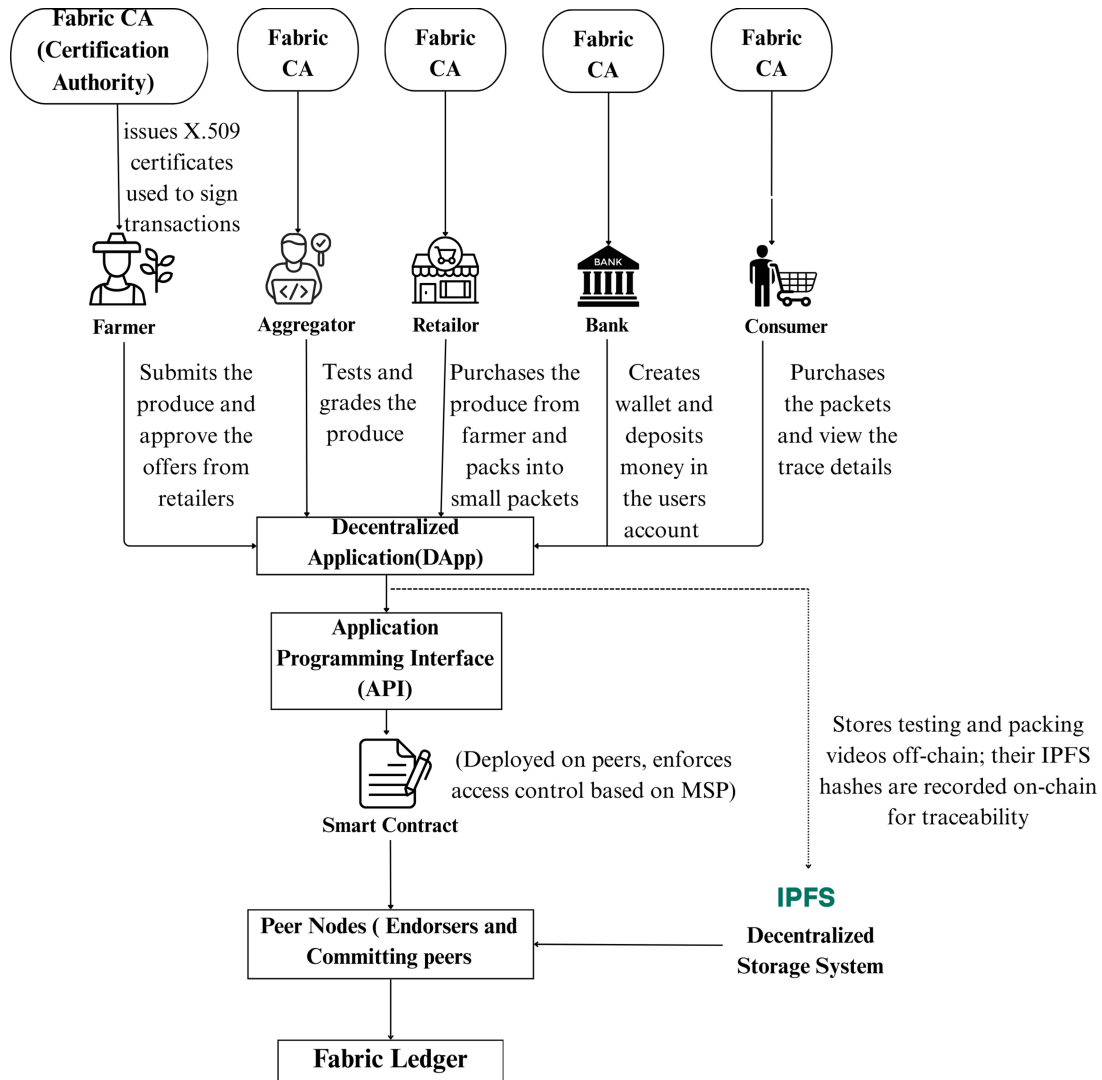


Figure 1: System overview of the proposed agricultural supply chain platform

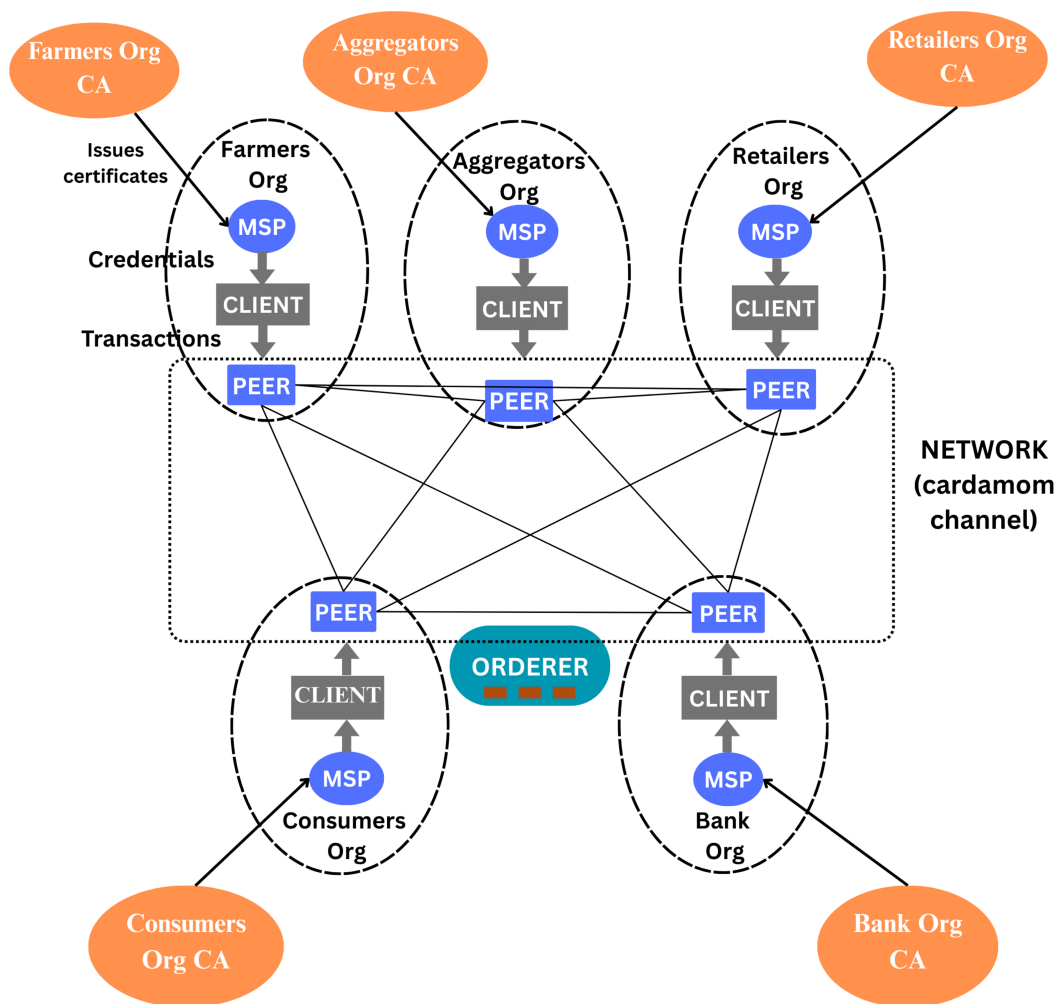


Figure 2: Network Architecture

are made available for retailer offers through `makeOffer()`, and farmers accept the highest valid offer using `acceptOffer()`. Retailers then pack accepted lots into packets using `packLotIntoPackets()`, while consumers purchase packets using `purchasePacket()` and verify provenance through lifecycle query functions. The overall sequence of stakeholder actions and smart contract functions is shown in Figure 3.

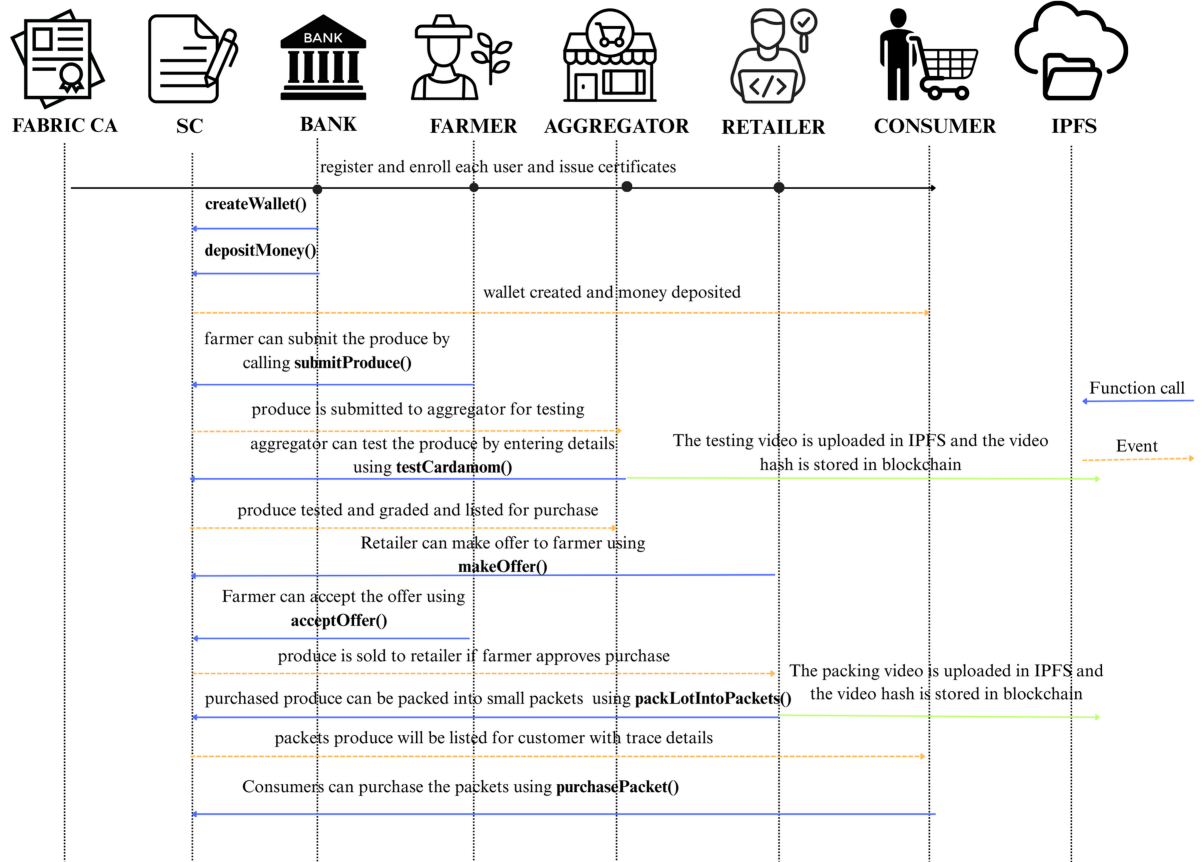


Figure 3: Sequence of stakeholder actions and smart contract functions in the agricultural supply chain

3.4 MVCC Conflict Vulnerabilities

In high-throughput environments, the unoptimized chaincode is highly susceptible to Multi-Version Concurrency Control (MVCC) conflicts. Hyperledger Fabric employs an optimistic concurrency model: transactions are executed, endorsed, and ordered before validation against the ledger state. During the validation phase, if the version of a state key read

during execution does not match the current committed version on the ledger, the transaction is marked invalid (MVCC `READ_CONFLICT`) and abruptly dropped. In this supply chain design, there are two primary architectural bottlenecks where these conflicts predominantly occur:

1. **Wallet Edits (*submitProduce*, *acceptOffer*, *purchasePacket*):** Functions that facilitate financial transactions inherently invoke a shared internal `_transfer` method. This method reads the current wallet balances of the sender and receiver, computes the new balances, and executes a `putState` operation to overwrite them. When multiple participants interact with a single entity concurrently (e.g., 50 farmers simultaneously submitting produce to a single aggregator), all 50 transactions read the exact same initial version of the aggregator’s wallet key. The first transaction to successfully commit will increment the ledger version of that wallet. Consequently, the remaining 49 transactions will fail validation, severely throttling throughput.
2. **Lot Highest Offer Edits (*makeOffer*):** The auction mechanism allows multiple retailers to bid on an available agricultural lot concurrently. The `makeOffer` function reads the lot’s current `highestOffer` parameter, validates if the new incoming bid is strictly greater, and overwrites the lot state. If multiple retailers bid on the exact same lot within the same block generation window, only the first endorsed transaction will successfully commit the updated lot state. The subsequent bids will encounter an MVCC conflict on the lot’s shared composite key, causing those transactions to fail regardless of whether their bid values were objectively higher.

These structural vulnerabilities form the foundational basis for the machine learning predictive models developed in this study, explicitly linking function complexity and hot-key density to deterministic transaction failure rates.

4 Methodology

4.1 Benchmarking Setup

The performance of the blockchain network was rigorously evaluated using Hyperledger Caliper, the industry-standard benchmarking tool for blockchain frameworks. The primary objective was to observe the dynamic shifts in transaction throughput and failure rates in response to increasing transaction send rates, fluctuating participant density, varying chain-code function complexity, and the resulting Multi-Version Concurrency Control (MVCC) hot-key contention.

4.1.1 Transaction Loads and Workload Functions

To capture the full spectrum of system performance—from an idle baseline to extreme network saturation—the experiments were conducted across a sequence of fixed-rate Transaction Per Second (TPS) loads: 1, 2, 4, 8, 10, 20, 50, 100, 200, 400, and 500 TPS. The default transaction duration for each load level was maintained at 5 seconds, with specific bulk

operations extended to 100 seconds to ensure steady-state measurements. The benchmarking suite tested multiple distinct chaincode functions to represent real-world supply chain workflows, including *testCardamom*, *makeoffer*, *acceptoffer*, *pack*, and *submitproduce*. Each of these functions possesses a different structural complexity in terms of the number of state reads and writes required, as well as the number of participants contending for the same ledger key.

4.1.2 Strategic Participant Matrix and Dynamic Caliper Network Generation

To evaluate the impact of concurrent user scaling without exhaustively testing redundant participant combinations, a strategic participant matrix was designed. The number of participants for key roles (e.g., Farmers, Aggregators, Retailers) was scaled across predefined thresholds: 1, 2, 5, 10, 15, 20, and 24.

For functions requiring interactive multi-party dependencies (such as Farmers submitting produce to Aggregators), a two-dimensional participant matrix was generated. This approach ensured comprehensive coverage of both balanced and imbalanced participant distributions while avoiding redundant reverse pairs.

Caliper workers were dynamically allocated based on the specific workload parameters. For example, in a *submitproduce* workload configured with 5 farmers and 24 aggregators (*submitproduce_f5_a24*), Caliper explicitly spawned 5 parallel worker processes acting as independent farmers. During execution, each farmer process probabilistically targeted one of the 24 available aggregators, actively stressing the shared ledger state and inducing measurable MVCC conflicts on the specific aggregator hot-keys.

To optimize the benchmarking infrastructure, un.js dynamically generated a workload-specific Caliper network configuration file for each experimental round. This dynamic provisioning ensured that only the required organizational identities (e.g., *User1* to *User24* for Farmers and Retailers) were actively instantiated and loaded into memory, thereby isolating chaincode performance metrics from artificial client-side resource starvation.

Table 1: Benchmark Workload Configurations and Participant Mapping

Workload Function	Primary Role	Caliper Workers	Participant Range
<i>submitproduce</i>	Farmers	X (number of farmers)	Farmers (1 to X), Aggregators (1 to Y)
<i>testCardamom</i>	Aggregators	N (number of aggregators)	Aggregators (1 to N)
<i>makeoffer</i>	Retailers	N (number of retailers)	Retailers (1 to N)
<i>acceptoffer</i>	Farmers	X (number of farmers)	Farmers (1 to X), Retailers (1 to Y)
<i>pack</i>	Retailers	N (number of retailers)	Retailers (1 to N)

4.2 Machine Learning Method and Procedure

The empirical output logs generated by Hyperledger Caliper across all benchmarking permutations were consolidated and extracted to formulate the primary dataset for the machine learning pipeline.

4.2.1 Static Analysis of Smart Contract Function Complexity

To precisely quantify the computational and state-interaction overhead of the chaincode, a custom static analysis pipeline (*analyzeComplexity*) was developed. This tool systematically parses the smart contract source code to extract deterministic operational metrics per function, including the total number of ledger reads (*getState*), ledger writes (*putState*), conditional branches, and iteration loops. Most importantly, the tool explicitly identifies the exact composite keys targeted during write operations (MVCC Write Keys). By defining these structural metrics (detailed in Table 2), the machine learning model can definitively correlate transaction performance with the exact number of highly contested hot-keys updated per invocation.

Table 2: Static Analysis of Smart Contract Function Complexity and MVCC Hot-Keys

Function	Total Reads	Total Writes	MVCC Write Keys (Hot-Keys)
<i>submitProduce</i>	3	3	Wallet Balance (fromKey, toKey)
<i>testCardamom</i>	1	1	None
<i>makeOffer</i>	2	1	lot.highestOffer
<i>acceptOffer</i>	3	3	Wallet Balance (fromKey, toKey)
<i>packLotIntoPackets</i>	1	$1 + n$	None
<i>purchasePacket</i>	3	3	Wallet Balance (fromKey, toKey)

*Note: For the empirical benchmarking simulations conducted in this study, the number of packets generated per lot (n) during the *packLotIntoPackets* function was fixed at 55.

4.2.2 Mechanics of MVCC Hot-Key Contention

In Hyperledger Fabric, Multi-Version Concurrency Control (MVCC) ensures data integrity by preventing concurrent transactions from modifying the exact same ledger state simultaneously. A “hot-key” bottleneck occurs when multiple active clients attempt to update the same state variable (key) at the exact same moment. If multiple transactions target the same state variable in the same block window, the first transaction successfully commits, while all subsequent concurrent transactions are instantly rejected by the validation peers with an MVCC read-write conflict, resulting in transaction failure.

In the context of the proposed agri-food smart contracts, these hot-key collisions mathematically dictate the failure rate and manifest in specific algorithmic patterns:

To provide a comprehensive dataset for the machine learning model, the benchmarking framework generated permutations across these features using the parameter ranges defined in Table 4.

4.2.3 Predictive Modeling and Baseline Comparison

To map the relationship between these system variables and the observed chaincode behavior, five distinct machine learning regression architectures were evaluated:

The selection of a Random Forest ensemble as the primary predictive architecture was driven by the unique failure mechanics of Hyperledger Fabric. Blockchain performance degradation—specifically MVCC read-write conflicts and throughput saturation—does not scale linearly. A

Table 3: Selected Features for Machine Learning Prediction

Feature	Description & Rationale
Transaction Send Rate (Load)	The targeted number of transactions submitted per second.
Number of Caliper Workers	The number of parallel client workers actively submitting transactions simultaneously, directly dictating concurrent thread pressure.
Participant Density	The explicit count of active Farmers, Aggregators, Retailers, Consumers, and Bank Users interacting with the smart contract.
Number of Ledger Writes	Extracted via static analysis, representing baseline structural complexity. Allows mathematical correlation of algorithmic topology to failure vulnerabilities rather than relying on categorical function names.
Hot-Key Participant Count	A derived metric quantifying severity of concurrent operations on shared state keys (e.g., competing retailers bidding on a single lot, or simultaneous wallet balance edits during settlements). The total count of participants interacting with these precise hot-keys directly mathematically dictates the probability of an MVCC read-write conflict and transaction failure.

Table 4: Experimental Benchmarking Parameter Ranges

Experimental Parameter	Evaluated Range / Values
Transaction Send Rate (TPS)	1, 2, 4, 8, 10, 20, 50, 100, 200, 400, 500
Number of Caliper Workers	1, 2, 5, 10, 15, 20, 24
Participant Density Matrix	(1,1), (1,2), (1,5), (2,2), (2,5), (5,5) ... up to (24,24)
Transactions per Round	500 fixed transactions per permutation
Workload Duration	Dynamic based on TPS, with specific caps (e.g., 100s)

network remains perfectly stable until a specific structural limit is hit (e.g., a critical mass of concurrent hot-key collisions or resource exhaustion), at which point performance drops off a severe "cliff" (jumping abruptly from 0% to high failure rates). Linear mapping models like Linear Regression and Support Vector Regression (SVR) are fundamentally incapable of modeling these strict, sudden thresholds. While Neural Networks (MLPs) can model non-linear boundaries, they inherently blur discrete integer transitions and lack interpretability. Conversely, tree-based ensemble methods like Random Forest create distinct conditional splits (e.g., IF Hot-Keys ≥ 2 AND Load $\geq 300 \rightarrow$ Failure = 85%), perfectly mimicking the conditional transaction validation logic of blockchain state machines. Furthermore, Random Forest natively calculates feature importance through independent bagging (Feature Importance), allowing for the extraction of mathematical proof regarding which structural parameters act as the primary bottlenecks, making it the optimal and most interpretable choice for this study.

Crucially, rather than predicting the absolute number of failed transactions (which is artificially biased by the total number of transactions sent), the models were trained to predict the precise *Failure Rate* (the percentage of failed transactions over the total submitted), alongside the

Table 5: Concurrent Hot-Key Contention Scaling Matrix across Participant Densities

<i>submitProduce</i> Hot-Keys	<i>makeOffer</i> Hot-Keys	<i>acceptOffer</i> Hot-Keys	<i>purchasePacket</i> Hot-Keys
2 (1 Farmer, 1 Aggregator)	1 Lot (1 Retailer competing)	2 (1 Farmer, 1 Retailer)	2 (1 Consumer, 1 Retailer)
8 (4 Farmers, 4 Aggregators)	1 Lot (4 Retailers competing)	8 (4 Farmers, 4 Retailers)	8 (4 Consumers, 4 Retailers)
20 (10 Farmers, 10 Aggregators)	1 Lot (10 Retailers competing)	20 (10 Farmers, 10 Retailers)	20 (10 Consumers, 10 Retailers)
30 (15 Farmers, 15 Aggregators)	1 Lot (15 Retailers competing)	30 (15 Farmers, 15 Retailers)	30 (15 Consumers, 15 Retailers)
40 (20 Farmers, 20 Aggregators)	1 Lot (20 Retailers competing)	40 (20 Farmers, 20 Retailers)	40 (20 Consumers, 20 Retailers)
48 (24 Farmers, 24 Aggregators)	1 Lot (24 Retailers competing)	48 (24 Farmers, 24 Retailers)	48 (24 Consumers, 24 Retailers)

Table 6: Evaluated Machine Learning Regression Architectures

Predictive Model	Architecture Class	Description & Rationale
Linear Regression	Baseline	Linear baseline; tested to prove incapability of modeling discrete, non-linear MVCC state ceilings.
Support Vector Regressor (SVR)	Distance-based	Evaluates structural mapping of continuous variables in high-dimensional space.
Multi-Layer Perceptron (MLP)	Neural Network	Capable of mapping non-linear boundaries but inherently blurs discrete integer thresholds and lacks feature interpretability.
Random Forest Regressor	Tree Ensemble	Aggregates multiple independent decision trees to reduce prediction variance.
a Random Forest ensemble	Boosting Ensemble	Sequentially builds trees to explicitly correct residual errors; natively provides transparent feature importance extraction.

successfully achieved system *Throughput*.

4.3 Data Collection and Feature Selection

The dataset’s features were carefully selected to capture the nonlinear interactions inherent in blockchain architecture that directly trigger transaction failures. Because MVCC failures in Hyperledger Fabric are predominantly caused by concurrent read/write operations on identical state keys, the selected independent variables explicitly quantified both the computational load and the specific state-write dependencies. The primary features included:

To provide a comprehensive dataset for the machine learning model, the benchmarking framework generated permutations across these features using the parameter ranges defined in Table 4.

5 Results and Discussion

5.1 Empirical Network Performance Analysis

Each core function was evaluated independently under varying send rates using Hyperledger Caliper. The tested functions included *submitProduce()*, *testCardamom()*, *makeOffer()*, *acceptOffer()*, *packLotIntoPackets()*, and *purchasePacket()*. The objective was to empirically quantify the architectural limits of the smart contracts prior to applying predictive machine learning.

Table 7: Selected Features for Machine Learning Prediction

Feature	Description & Rationale
Transaction Send Rate (Load)	The targeted number of transactions submitted per second.
Number of Caliper Workers	The number of parallel client workers actively submitting transactions simultaneously, directly dictating concurrent thread pressure.
Participant Density	The explicit count of active Farmers, Aggregators, Retailers, Consumers, and Bank Users interacting with the smart contract.
Number of Ledger Writes	Extracted via static analysis, representing baseline structural complexity. Allows mathematical correlation of algorithmic topology to failure vulnerabilities rather than relying on categorical function names.
Hot-Key Participant Count	A derived metric quantifying severity of concurrent operations on shared state keys (e.g., competing retailers bidding on a single lot, or simultaneous wallet balance edits during settlements). The total count of participants interacting with these precise hot-keys directly mathematically dictates the probability of an MVCC read-write conflict and transaction failure.

5.1.1 Throughput and Latency Scalability

As visualized in Figure 4 and Figure 5, the network’s throughput scaling is directly dictated by the algorithmic complexity of the invoked function. Under low concurrent loads (50-100 TPS), all functions scale near-linearly. However, as the transaction send rate accelerates toward 500 TPS, severe performance divergence occurs. Lightweight functions requiring minimal state modifications, such as *makeOffer()* and *testCardamom()*, demonstrate exceptional resilience. At 500 TPS, *makeOffer()* achieved the best scalability among the tested write functions, maintaining very low average latency and reaching a maximum throughput of over 400 TPS. This result reflects its highly optimized design, requiring only a single ledger write to update the lot’s highest offer parameter.

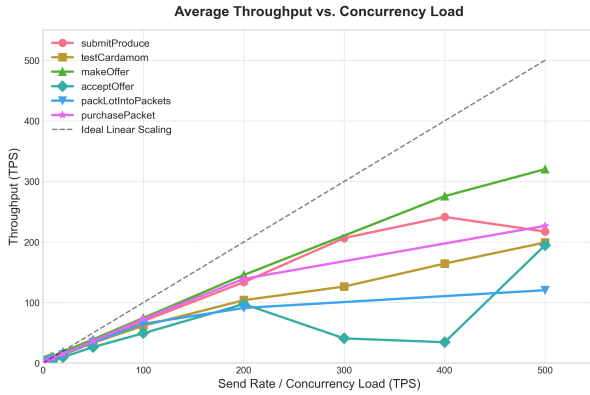


Figure 4: Average Throughput across varying transaction loads.

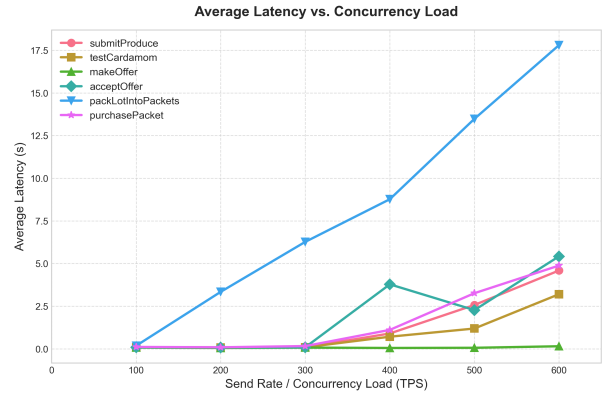


Figure 5: Average Latency across varying transaction loads.

5.1.2 Impact of Ledger Write Complexity

Conversely, functions with heavier structural payloads exhibited aggressive performance ceilings. As demonstrated in Figure 6 and Figure 7, there is a strict inverse correlation between Ledger Write Complexity and sustained throughput at peak loads. The *packLotIntoPackets()* function—which executes an intensive $1 + n$ sequential ledger writes to generate individual produce packets—hit a hard throughput ceiling of approximately 140 TPS. The sheer volume of key-value writes per transaction overwhelmed the Fabric endorsement and commit phases, causing average latency to spike exponentially compared to the simpler functions.

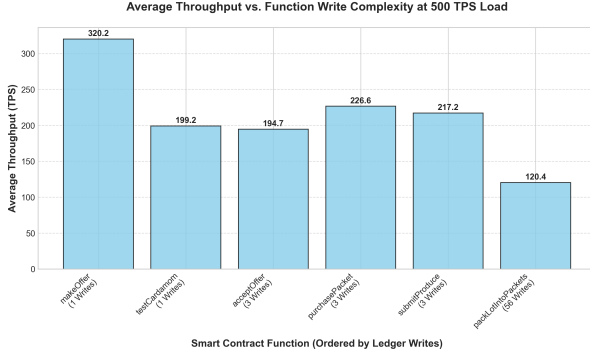


Figure 6: Average Throughput vs. Write Complexity at 500 TPS.

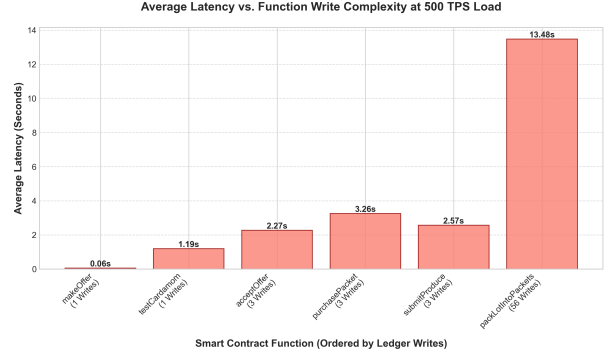


Figure 7: Average Latency vs. Write Complexity at 500 TPS.

5.1.3 MVCC State Contention Limits

Finally, intermediate functions requiring complex financial settlements between two distinct participants (*submitProduce()*, *acceptOffer()*, and *purchasePacket()*) sat firmly in the middle of the performance spectrum. These functions generally capped between 200 and 250 TPS. While they required fewer raw ledger writes than the packing function, their reliance on overlapping participant wallet updates caused severe Multi-Version Concurrency Control (MVCC) read-write conflicts on shared assets, forcing the validation peers to reject massive numbers of concurrent transactions.

To empirically validate the structural bottlenecks defined in Table 5, Figure 12 presents the recorded transaction failure rates plotted directly against the number of concurrent hot-key participants for four highly contended functions. As demonstrated, while higher overall transaction loads (TPS) elevate the baseline failure floor, the steepness of the failure curve is overwhelmingly driven by the specific number of participants simultaneously attempting to update a shared ledger state.

5.2 Predictive Benchmarking

While empirical benchmarking identifies the hard performance ceilings of the supply chain smart contracts, transaction failures within permissioned blockchains are driven by complex, non-linear interactions between transaction load, participant counts, ledger read/write complexities, and network conditions. To mathematically map these failure triggers, we extracted the Hyperledger Caliper transaction logs and trained predictive machine learning models. The models were trained on a strategic participant matrix (1 to 24 peers) under varying fixed-rate TPS loads (1 to 500) across all primary chaincode functions.

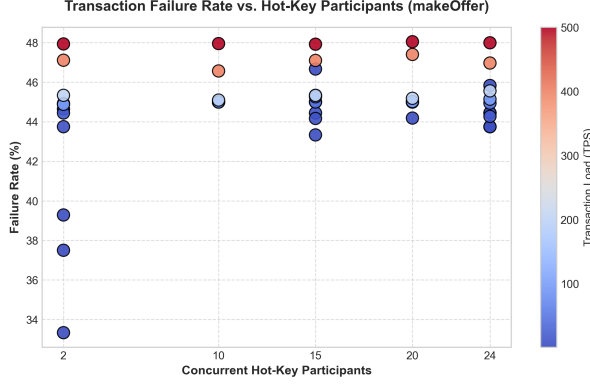


Figure 8: makeOffer

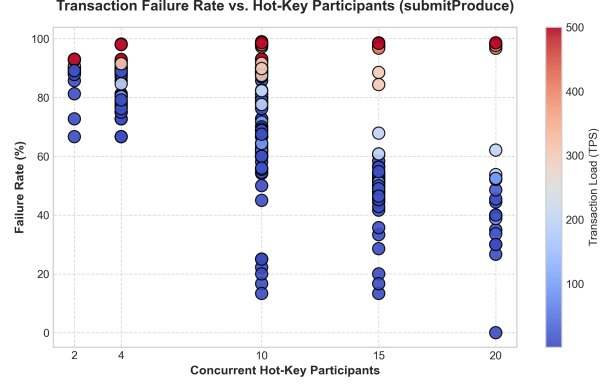


Figure 9: submitProduce

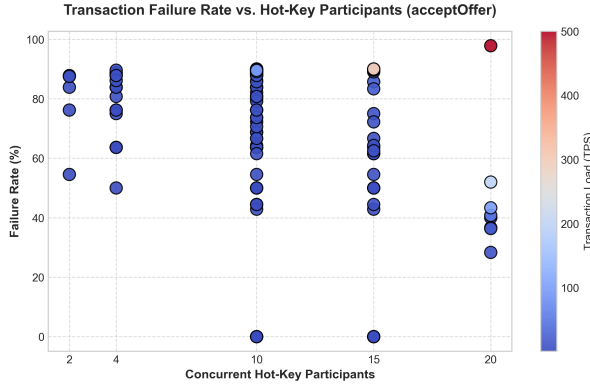


Figure 10: acceptOffer

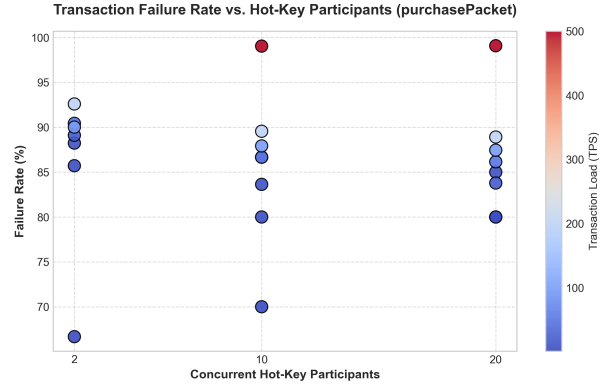


Figure 11: purchasePacket

Figure 12: Empirical transaction failure rates scaling in direct proportion to the number of concurrent hot-key participants.

5.2.1 Evaluated Predictive Architectures

To validate the hypothesis that MVCC conflicts scale non-linearly and exhibit strict threshold ”cliffs” based on structural topology, four regression architectures were evaluated: Linear Regression, Gradient Boosting, Random Forest, and a Random Forest ensemble. The empirical performance of these predictive architectures was evaluated for both successful transaction throughput and the transaction failure rate.

As mathematically proven by the R^2 scores across both prediction targets, the baseline linear architecture (Linear Regression) failed to accurately model the non-linear distributions, particularly for failure rates ($\sim 0.61 R^2$). This explicitly validates the hypothesis that blockchain MVCC saturation induces severe, non-linear performance cliffs rather than gradual linear degradation. Conversely, tree-based ensemble architectures (Random Forest) captured over 92% of the variance for both throughput and failure rates. **Random Forest** was explicitly selected as the primary analytical framework for this study. Random Forest’s independent bagging mechanism provides a more balanced and robust distribution of feature importance across highly correlated variables, avoiding the aggressive, single-feature dominance often exhibited by sequential boosting models.

5.2.2 Throughput Prediction

Table 8 details the empirical performance of the predictive architectures when tasked with modeling the network’s Transaction Throughput.

Table 8: Comparative Predictive Performance (Target: Transaction Throughput)

Machine Learning Model	R^2 Score	MAE	RMSE
Linear Regression	0.8140	17.497	30.257
Gradient Boosting	0.9142	8.339	20.541
Random Forest Regressor	0.9298	6.001	18.580

The trained Random Forest model demonstrated robust generalization for throughput prediction. As shown in Figure 13, the model confirmed that the raw **Transaction Send Rate (Load)** is the overwhelming constraint on successful throughput, absorbing an incredible 87.8% of the predictive importance. Unlike failure events, successful throughput is primarily dictated by raw network pressure rather than structural smart contract complexity.

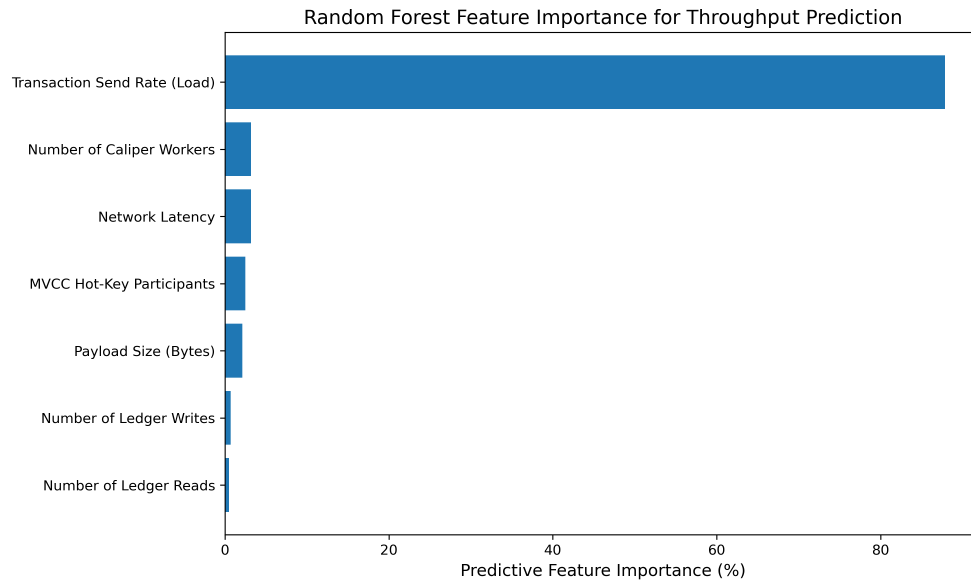


Figure 13: Random Forest Feature Importance ranking the dominant variables that constrain successful transaction throughput.

5.2.3 Failure Prediction and Feature Importance

Table 9 details the empirical performance of the architectures when tasked with modeling the Transaction Failure Rate.

The trained Random Forest model successfully mapped the complex, multi-factorial catalysts responsible for transaction timeouts and Multi-Version Concurrency Control (MVCC) **READ.CONFLICT** errors. As shown in Figure 14, transaction failures are driven by a distributed combination of state-contention and network stress. **MVCC Hot-Key Participants** (23.1%), raw **Transaction Load**

Table 9: Comparative Predictive Performance (Target: Transaction Failure Rate)

Machine Learning Model	R^2 Score	MAE	RMSE
Linear Regression	0.6180	0.158	0.216
Gradient Boosting	0.9094	0.070	0.105
Random Forest Regressor	0.9602	0.032	0.069

(20.7%), **Number of Ledger Reads** (18.8%), and **Payload Size** (17.1%) collectively dictate the failure rate. This balanced feature distribution mathematically proves that MVCC failures cannot be attributed to a single bottleneck; rather, they occur when dense participant state-traversals collide simultaneously with high network data volumes.

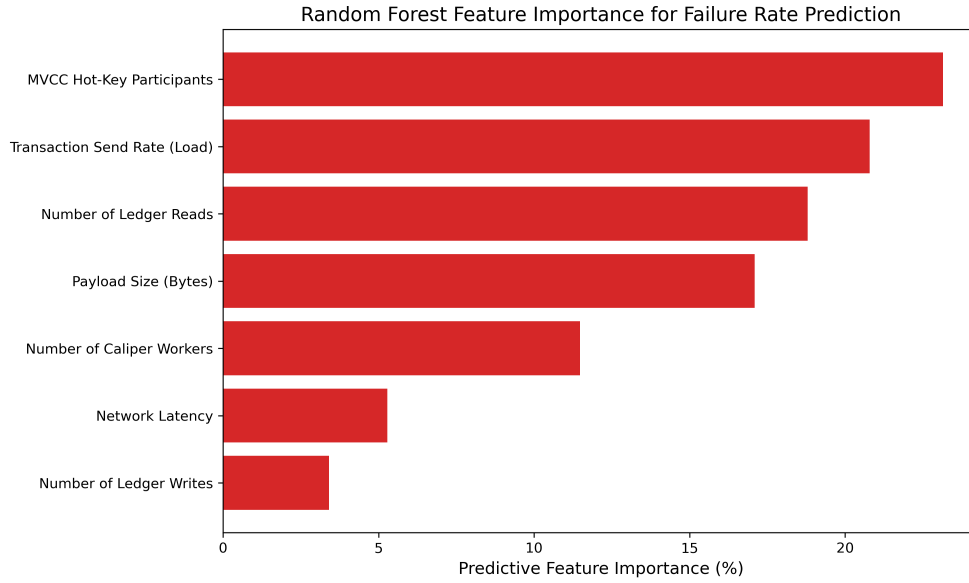


Figure 14: Random Forest Feature Importance ranking the distributed variables that trigger transaction failures.

To understand *how* these variables push the Hyperledger orderer and endorser nodes toward failure, we computed SHapley Additive exPlanations (SHAP) values. Figure 15 presents the SHAP summary plot, which explicitly proves the directional relationship: high transaction loads (red dots) heavily push the model toward predicting a higher failure rate (positive SHAP value), whereas lightweight transactions with low participant counts push the model toward a 100% success rate (negative SHAP value).

The SHAP Dependence plot (Figure 16) further highlights the severe non-linear bottlenecks present in the unoptimized smart contract architecture. As concurrent TPS increases, the likelihood of a transaction encountering a stale read or timeout increases exponentially. This predictive modeling validates the necessity for architectural refactoring—such as implementing append-only partial composite keys—to decouple state writes and mitigate concurrent hot-key collisions.

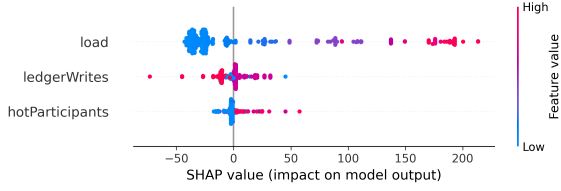


Figure 15: SHAP Summary Plot demonstrating the directional impact of concurrent load and function complexity on transaction failures.

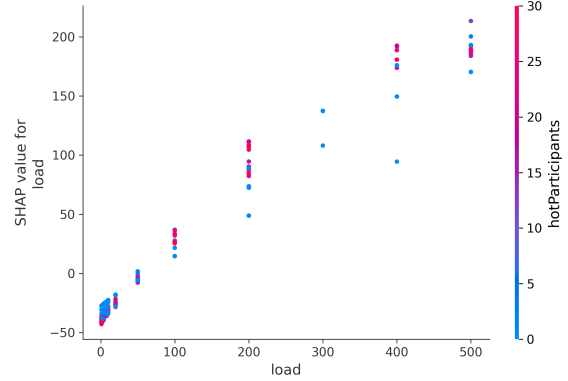


Figure 16: SHAP Dependence Plot highlighting the non-linear interaction between load and participant counts.

6 Conclusion

This study set out to address a fundamental challenge in the practical deployment of permissioned blockchain systems: the difficulty of predicting and diagnosing non-linear Multi-Version Concurrency Control (MVCC) failures before they cause catastrophic throughput degradation in production environments. Using a Hyperledger Fabric-based agricultural supply chain system as a real-world empirical testbed, we developed and validated a machine learning framework—based on a Random Forest ensemble—that accurately forecasts both MVCC failure rates and transaction throughput from interpretable structural and operational features.

The performance analysis, conducted via Hyperledger Caliper across 11 send-rate levels and a comprehensive participant matrix, revealed severe MVCC-induced throughput degradation in functions with shared wallet keys (`extitsubmitProduce`, `extitacceptOffer`, `extitpurchasePacket`) and shared lot state (`extitmakeOffer`). Critically, the Random Forest predictive model demonstrated that failures are a distributed phenomenon driven by a combination of `extbfMVCC Hot-Key Participants` (23.1%), `extbfTransaction Load` (20.7%), and `extbfNumber of Ledger Reads` (18.8%). Conversely, for throughput prediction, the `extbfTransaction Send Rate (Load)` was the overwhelming constraint, absorbing an incredible 87.8% of the predictive importance. These findings mathematically prove that while throughput is dictated by raw network pressure, MVCC failures occur when dense participant state-traversals collide simultaneously with high network data volumes.

The results confirm that append-only, composite-key-based chaincode designs substantially reduce MVCC conflicts, and that the Random Forest model provides an accurate, interpretable diagnostic tool for identifying which chaincode functions and participant configurations are at highest risk. Looking ahead, future work will extend this predictive framework to distributed multi-host deployments with wide-area network conditions, explore dynamic chaincode optimization strategies guided by model predictions, and integrate real-time IoT sensor data streams for objective quality grading. The supply chain finance layer can be extended with weather-indexed smart contract insurance and credit access mechanisms leveraging verified on-chain transaction histories, further strengthening the financial resilience and scalability of agri-food blockchain deployments.

Code Availability

The source code, configuration files, and benchmarking scripts used in this study are publicly available at:

<https://github.com/dhanishjose123/fabric-traceability-cardamom>

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used **DeepL** and **ChatGPT** in order to improve language and readability. After using these tools, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRedit authorship contribution statement

Dhanish Jose: Writing – original draft, Writing – review & editing, Validation, Methodology, Conceptualization.

Pradeepmon T G: Writing – review & editing, Validation, Resources, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by **National Institute of Technology Calicut (NIT Calicut)**.

References

- Devi, M. B., Singh, N. C., and Singh, T. B. (2025). Dynamic sharding with ai-driven load balancing for blockchain networks. *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*.
- Dewi, D. A. (2025). Predicting throughput and latency in hyperledger fabric blockchains using random forest regression. *Journal of Current Research in Blockchain*.
- El Hajji, M., Es-saady, Y., Ait Addi, M., and Antari, J. (2024). Optimization of agrifood supply chains using Hyperledger Fabric blockchain technology. *Computers and Electronics in Agriculture*, 227:109503.

- Iftekhar, A. and Cui, X. (2021). Blockchain-Based Traceability System That Ensures Food Safety Measures to Protect Consumer Safety and COVID-19 Free Supply Chains. *Foods*, 10(6):1289.
- Joo, J. and Han, Y. (2021). An Evidence of Distributed Trust in Blockchain-Based Sustainable Food Supply Chain. *Sustainability*, 13(19):10980.
- Kaushal, R. K., Kumar, N., Boonchieng, E., Mapari, S., Kukreja, V., and Verma, A. (2025). Convergence of blockchain and IoT for managing decentralized medical records. *Scientific Reports*, 15(1):44512.
- Kumar, H., Kaushal, R. K., Kumar, N., Verma, A., and Boonchieng, E. (2026). Integrating transparency and privacy in grievance redressal through Hyperledger Fabric with multi-organization support. *Scientific Reports*, 16(1):5574.
- Marchese, A. and Tomarchio, O. (2022). A Blockchain-Based System for Agri-Food Supply Chain Traceability Management. *SN Computer Science*, 3:279.
- Nayal, K., Raut, R. D., Narkhede, B. E., Priyadarshinee, P., Panchal, G. B., and Gedam, V. V. (2021). Antecedents for blockchain technology-enabled sustainable agriculture supply chain. *Annals of Operations Research*, 327(1):293–337.
- Pawar, M. K. and Patil, P. (2025). Scalability enhancement for blockchains by dynamic difficulty level adjustment: A machine learning approach. *Engineering, Technology & Applied Science Research*.
- Rani, P., Sharma, P., Gupta, I., and Gaba, P. (2025). STP: a permissioned blockchain solution for dynamic pricing and traceability in food supply chain. *Peer-to-Peer Networking and Applications*, 18(4):224.
- Shahid, A., Almogren, A., Javaid, N., Al-Zahrani, F., Zuair, M., and Alam, M. (2020). Blockchain-based agri-food supply chain: a complete solution. *IEEE Access*, 8:69230–69243.
- Stoltidis, A., Choumas, K., and Korakis, T. (2024). Performance optimization of high-conflict transactions within the hyperledger fabric blockchain.
- Talib, M. A., Masmoudi, M., Nasir, Q., Abbas, S., Obeid, M. H., and Mokhamed, T. (2026). Blockchain-enabled halal food certification and supply chain framework in the UAE. *Results in Engineering*, 29:109740.
- Thakkar, P., Nathan, S., and Viswanathan, B. (2018). Performance Benchmarking and Optimizing Hyperledger Fabric Blockchain Platform. In *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 264–276. ISSN: 2375-0227.
- Trabelsi, H. and Zhang, K. (2023). Early detection for multiversion concurrency control conflicts in hyperledger fabric.
- Weerabahu, S., Samaranayake, P., Dasanayaka, S., and Wickramasinghe, C. (2022). Challenges of agri-food supply in city region food systems: an emerging economy perspective. *Journal of Agribusiness in Developing and Emerging Economies*, 12(2):161–182.

- World Wildlife Fund (WWF) (2019). The WWF backs blockchain to unpick messy food supply chains. Wired UK. <https://www.wired.com/story/food-supply-chains-blockchain-wwf/> (Accessed: 22 August 2025).
- Zhang, X., Li, Y., Peng, X., Zhao, Z., Han, J., and Xu, J. (2022). Information Traceability Model for the Grain and Oil Food Supply Chain Based on Trusted Identification and Trusted Blockchain. *International Journal of Environmental Research and Public Health*, 19(11):6594.
- Zhang, Y., Wu, X., Ge, H., Jiang, Y., Sun, Z., Ji, X., Jia, Z., and Cui, G. (2023). A Blockchain-Based Traceability Model for Grain and Oil Food Supply Chain. *Foods*, 12(17):3235.
- Zhao, G., Liu, S., Lopez, C., Lu, H., Elgueta, S., Chen, H., and Boshkoska, B. (2019). Blockchain technology in agri-food value chain management: A synthesis of applications, challenges and future research directions. *Computers in Industry*, 109:83–99.